SCALABILITY, PERFORMANCE, AND CAPACITY PLANNING

Engineering Case Study for Enterprise Data and AI Systems

Author	Venkata Lakshmi Parimala Pasupuleti
Primary domains	Affordable Housing, enterprise data systems, production AI
Systems represented	Income limits, RabbitMQ workflows, documents, migrations, APIs, LeaseI, AgentProbe
Document type	Technical engineering case study
Direct experience	SQL optimization, batch processing, production operations, asynchronous workflows, data validation
Target guidance	Modern capacity modeling, distributed scaling, cloud-native operations, AI cost controls
Design principle	Scale only after measuring the real constraint
Status	Portfolio documentation based on real engineering situations

This document separates directly demonstrated engineering practices from forward-looking modernization. It does not claim that every target scaling mechanism was deployed in the original production systems.

Document Map

1. Executive Summary
2. Scope and Evidence Boundaries
3. Performance as a Business Property
4. Workload Characterization
5. Capacity Planning Model
6. Database Performance Engineering
7. ETL and Large-Scale Transformations
8. Asynchronous Processing with RabbitMQ
9. API and Service Performance
10. Caching and Read Optimization
11. Document and Object-Storage Workloads
12. AI Runtime Performance and Cost
13. Retrieval and Vector-Search Scaling
14. Concurrency and Resource Management
15. Reliability Under Load
16. Observability and Performance Metrics
17. Performance Testing
18. Incident Diagnosis and Bottleneck Analysis
19. Deployment and Scaling Strategy
20. Cost Efficiency
21. Trade-offs and Anti-Patterns
22. Modernization Roadmap
23. Reusable Capacity Checklists
24. Lessons Learned

1. Executive Summary

Performance engineering is the discipline of keeping a system responsive, correct, and economically sustainable as demand grows. In the systems I worked on, scale did not mean only millions of web requests. It included annual source data expanding into approximately 180,000 derived records, thousands of rent-related updates, document-generation workloads, long-running database migrations, support investigations that crossed multiple systems, and AI workflows that combined retrieval, tools, and model calls.

My practical approach is measurement first. I begin with business demand, data volume, service time, downstream dependencies, and failure impact. I then identify the dominant constraint: database access, transformation logic, serialization, network calls, queue backlog, document I/O, or model latency. The solution should target that constraint without introducing unnecessary infrastructure.

This case study connects the performance lessons from SQL optimization, ETL ownership, RabbitMQ processing, production migrations, FastAPI services, LeaseI, and AgentProbe. It also describes how I would extend those principles into capacity forecasting, horizontal scaling, workload isolation, AI cost management, and production performance gates.

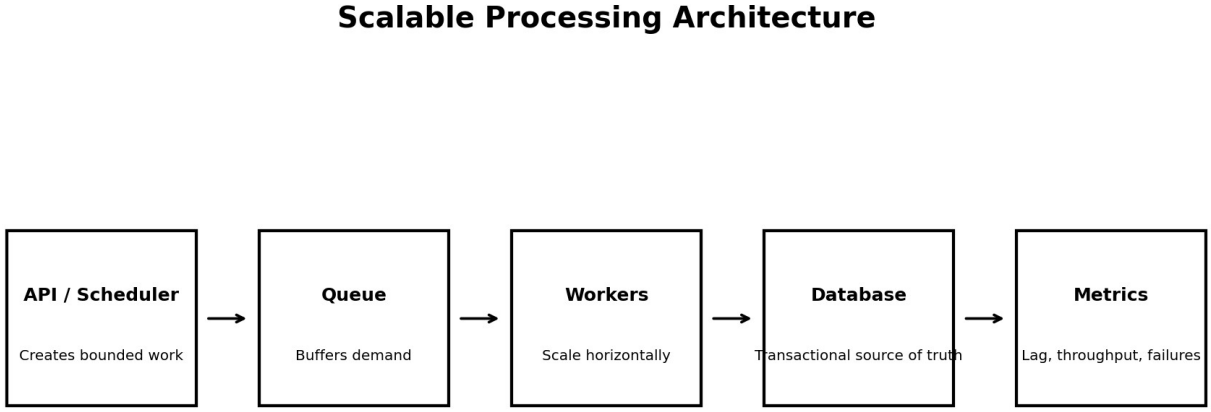


Figure 1. A scalable processing path that separates intake, buffering, execution, state, and measurement.

2. Scope and Evidence Boundaries

The directly demonstrated experience in this document includes complex SQL and stored procedures, execution-plan analysis, annual data transformations, staging and reconciliation, asynchronous RabbitMQ processing, production release validation, AWS S3 integration, synchronous LeaseI orchestration, and local AI evaluation. Modern distributed infrastructure is presented as a target design where it was not part of the original implementation.

Area	Directly demonstrated	Forward-looking guidance
Database	Query tuning, stored procedures, indexing decisions, data reconciliation, migration validation	Read replicas, partition lifecycle, connection-pool governance, database autoscaling
Batch and ETL	Annual imports, staging, expansion, validation, publication	Workflow engine, elastic workers, object-based ingestion, lineage service
Messaging	RabbitMQ-based long-running processing and worker concepts	Autoscaled consumers, formal DLQs, outbox pattern, SLO-based lag control
APIs	REST/FastAPI endpoints and synchronous investigation workflows	Horizontal stateless scaling, rate limits, backpressure, distributed tracing
AI	LangGraph workflows, retrieval, tools, local models, evaluation	Model gateways, semantic caches, multi-tenant vector scaling, cost budgets

3. Performance as a Business Property

A system is not fast in isolation. It is fast enough for a specific user or operational need. An annual import can run for several minutes if it is controlled, observable, and completed before publication. A support investigation may need an answer within seconds because a customer is blocked. A document-generation request may need immediate acknowledgment even if rendering finishes asynchronously.

- Define latency and throughput targets per workflow rather than using one platform-wide number.
- Include correctness and recoverability in the performance objective.
- Measure the business effect of backlog, such as delayed certifications or unresolved tickets.
- Distinguish interactive, batch, asynchronous, and analytical workloads.
- Treat cost as a capacity constraint, especially for AI and external APIs.

The strongest performance decision is often moving work out of the user request. RabbitMQ was valuable because long-running gross-rent updates could be buffered and processed independently instead of holding a synchronous request open.

4. Workload Characterization

Capacity planning starts with a workload model. Before selecting infrastructure, I want to know what enters the system, how often, how large it is, how it expands, what state it touches, and what deadline it must meet.

Workload	Primary scale dimension	Important constraint
Annual HUD import	Source rows, derived rows, jurisdictions, household sizes	Transformation time and data integrity

Workload	Primary scale dimension	Important constraint
Gross-rent processing	Number of affected households and properties	Queue backlog and worker throughput
Document generation	Templates, merge fields, generated files	Database reads, rendering time, storage I/O
Database migration	Rows changed, locks, index maintenance	Transaction duration and rollback window
Support investigation	Sources queried, historical tickets, tool calls	Interactive latency and evidence quality
AI evaluation	Test cases, models, prompts, traces	Model throughput, token cost, reproducibility

Workload shape matters as much as average volume. Annual data releases create predictable bursts. Customer incidents create unpredictable high-priority demand. Retries can multiply workload after a dependency failure. A design that handles average demand but collapses during synchronized bursts is not production-ready.

5. Capacity Planning Model

A practical capacity model connects demand, service time, concurrency, backlog, and cost. The goal is not to forecast every detail perfectly. The goal is to identify when the current design will stop meeting its objectives and which resource will fail first.

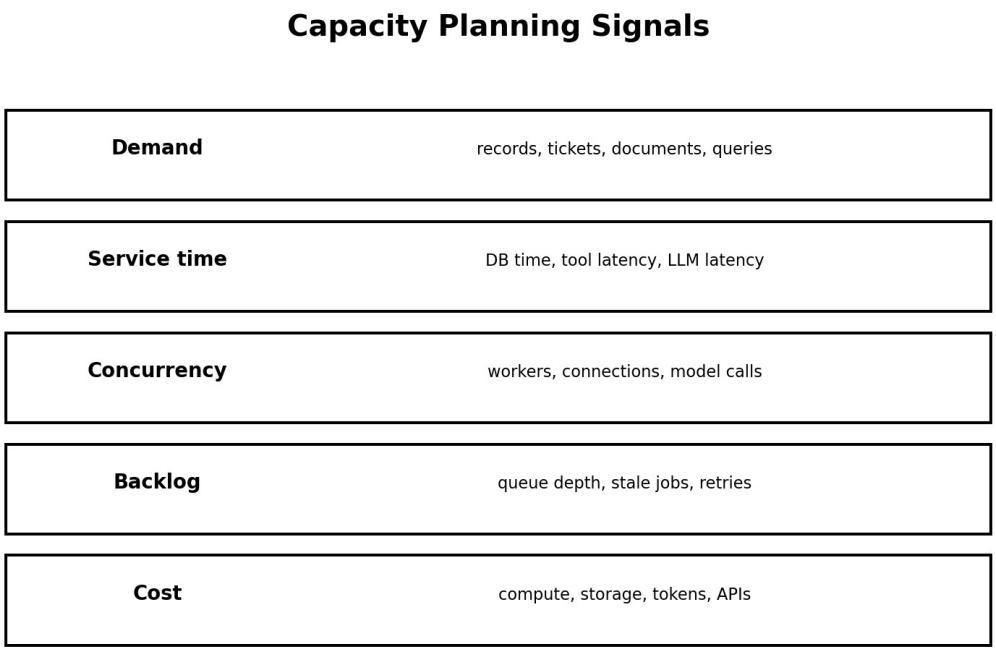


Figure 2. Capacity planning depends on demand, service time, concurrency, backlog, and cost.

- Arrival rate: how many requests, records, jobs, documents, or evaluations enter per unit of time.

- Service time: how long one unit occupies a database connection, worker, CPU core, model endpoint, or external API.
- Concurrency: how many units can execute safely without exhausting shared resources.
- Backlog tolerance: how much queued work is acceptable before the business deadline is missed.
- Headroom: reserved capacity for failures, releases, retries, and unexpected bursts.
- Cost ceiling: the spend limit for compute, storage, external APIs, and model tokens.

For queue-based processing, a simple planning check is whether worker throughput remains greater than sustained arrival rate. If not, backlog will grow indefinitely. During bursts, the key metric is time to drain: backlog divided by net processing capacity.

6. Database Performance Engineering

The relational database was central to many of the production systems I supported. Database performance cannot be improved safely by guessing. The process is to identify the slow business operation, inspect the query plan and row estimates, understand data distribution, change one factor, and validate both speed and output correctness.

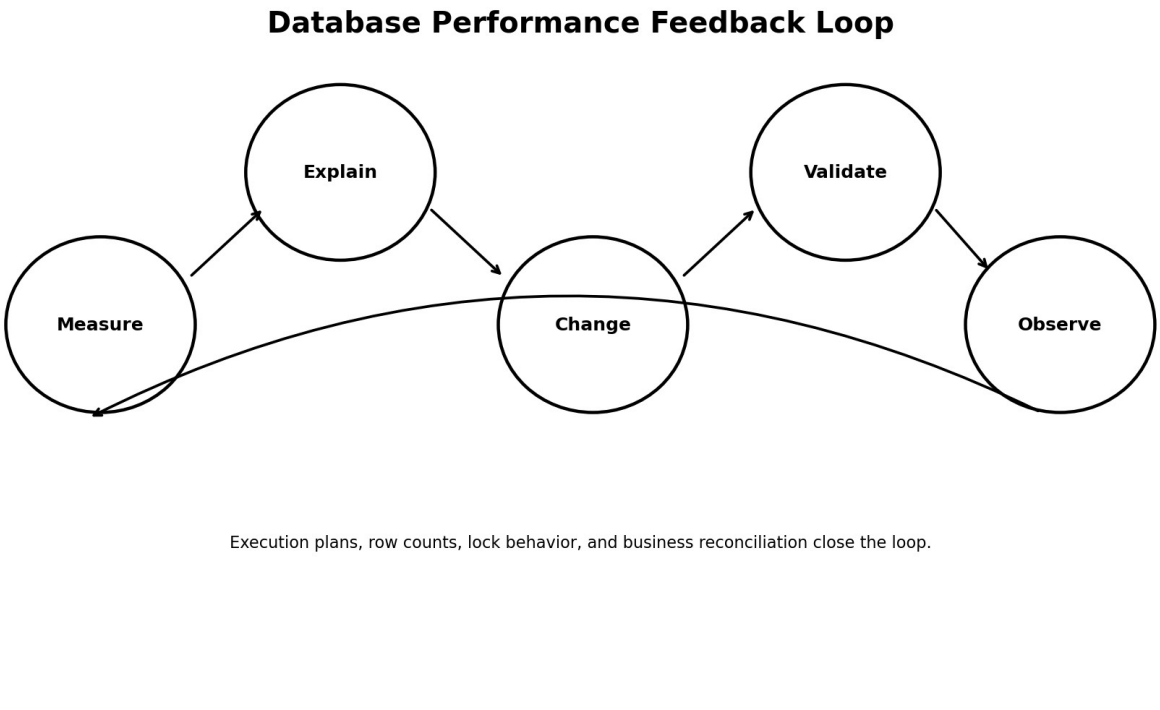


Figure 3. Database performance improvement is an evidence-based feedback loop.

6.1 Execution plans

I examine scan types, join order, row-estimate error, sort and aggregation cost, repeated subplans, and whether predicates are applied early. A slow query can be caused by missing access paths, but it can also be caused by poor cardinality estimates, non-sargable predicates, excessive intermediate rows, or the wrong transaction shape.

6.2 Index decisions

Indexes should support proven access patterns. Each index has write, storage, and maintenance cost. I prefer targeted composite indexes aligned with selective predicates and join columns rather than indexing every frequently mentioned field.

6.3 Stored procedures

Stored procedures were effective for set-based transformations close to the data. Their trade-off is concentrated complexity. They require versioning, representative tests, clear transaction boundaries, and operational visibility.

6.4 Connection and lock pressure

A query can be individually fast and still harm the platform if too many copies run concurrently. Capacity planning must include connection-pool size, lock duration, transaction scope, and the effect of batch workers on interactive traffic.

6.5 Business validation

Performance changes must preserve row counts, totals, eligibility results, and effective-date behavior. A faster query that changes business meaning is a failure.

Symptom	Likely investigation	Possible response
High CPU	Plans, full scans, repeated expressions, excessive rows	Rewrite query, improve selectivity, add justified index
High I/O	Large scans, poor clustering, repeated reads	Access-path change, batching, caching, archival
Lock waits	Long transactions, broad updates, competing workflows	Smaller batches, shorter transactions, workload isolation
Connection exhaustion	Slow requests, excessive concurrency, leaked connections	Pool limits, timeouts, backpressure, query optimization
Plan regression	Data distribution or statistics change	Refresh statistics, stabilize query, compare plans

7. ETL and Large-Scale Transformations

The annual income-limit workflow expanded a relatively small published source into a much larger domain model. That expansion created scale through combinations of geography, household size, percentage, effective date, and downstream business representation. The design used staging, validation, transformation, and controlled publication rather than modifying live records directly.

1. Land the source in a staging area with source identity and processing version.
2. Validate schema, required fields, duplicates, jurisdiction mapping, and numeric ranges.
3. Transform in set-based operations where practical.
4. Write derived records in bounded transactions.
5. Reconcile expected and actual counts at each stage.

6. Publish only after validation gates pass.
7. Retain enough history and evidence to support rollback and later investigation.

Batch size is a trade-off. Larger batches reduce overhead but increase lock duration, rollback cost, and memory pressure. Smaller batches improve recoverability and coexistence but add scheduling and checkpoint complexity. The correct size should be derived from measurements in a production-like environment.

8. Asynchronous Processing with RabbitMQ

RabbitMQ separates demand from execution. The producer can acknowledge that work was accepted while consumers process units independently. This improves responsiveness and provides a natural backlog, but it also creates new capacity concerns: queue depth, consumer throughput, retries, dead letters, duplicate delivery, and downstream database saturation.

Component	Performance responsibility
Producer	Create bounded messages, avoid oversized payloads, publish at a controlled rate
Broker	Persist messages as required, expose queue depth and delivery rate, protect memory and disk
Consumer	Process idempotently, acknowledge only after durable success, limit concurrency
Database	Support aggregate worker load without harming interactive requests
Retry path	Delay and cap retries so failures do not become a retry storm
Operations	Monitor lag, oldest-message age, failure rate, and time to drain

More workers do not automatically improve throughput. If the database is the bottleneck, adding consumers can increase lock waits and reduce total throughput. Consumer concurrency should be increased gradually while observing the complete system, not only the queue.

9. API and Service Performance

Interactive APIs should perform only the work required to produce a safe response. Long-running ingestion, rendering, evaluation, or bulk correction should move to asynchronous execution. Stateless APIs can scale horizontally, but only if shared state, database connections, caches, and external dependencies are also designed for concurrency.

- Set explicit timeouts for database and external calls.
- Use bounded retries only for transient and idempotent operations.
- Apply request limits and rate controls to protect expensive paths.
- Return an operation identifier for long-running work.
- Use pagination and server-side filtering for large result sets.
- Propagate correlation identifiers through services and queues.
- Distinguish user latency from backend completion time.

FastAPI is a suitable application layer for LeaseI because it supports typed validation and asynchronous I/O. The current project runs investigations synchronously, which is acceptable at demo scale. A production version would submit long investigations to workers and expose status without holding the request open.

10. Caching and Read Optimization

Caching can reduce latency and external cost, but it introduces staleness and invalidation risk. In regulated workflows, the cache key must include the business dimensions that define correctness, such as year, jurisdiction, household size, AMI percentage, property, or configuration version.

Candidate	Cache value	Main risk
Published HUD limits	Reduces repeated external or database lookup	Using stale year or jurisdiction
Product documentation retrieval	Reduces repeated parsing and embedding	Serving superseded release guidance
Historical incident search	Speeds repeated support investigations	Missing newly confirmed root cause
Model or embedding response	Reduces AI latency and cost	Returning answer for mismatched tenant/context
Property settings	Can reduce repeated reads	Configuration changes not reflected immediately

I would not cache sensitive customer-specific answers broadly. Cache scope should match authorization scope, and invalidation should be tied to source version or change events where possible.

11. Document and Object-Storage Workloads

Document workflows combine metadata queries, merge-field resolution, rendering, object transfer, and storage. Performance bottlenecks can arise from repeated database lookups, large file I/O, serial processing, or metadata inconsistency that triggers retries and regeneration.

- Batch metadata reads where possible instead of querying per merge field.
- Stream large objects rather than loading entire files into memory.
- Separate document metadata from object transfer so each can scale independently.
- Use checksums and size validation to avoid repeated full-content inspection.
- Limit concurrent migrations to protect database and storage dependencies.
- Track generation and migration backlog by age, not only count.

During the AWS S3 migration, module-specific validation was more important than maximizing raw copy speed. A fast migration that loses ownership metadata or makes documents unavailable through the product is unsuccessful.

12. AI Runtime Performance and Cost

AI performance is a pipeline property. Retrieval, reranking, tool calls, model generation, validation, and escalation each contribute latency and failure probability. Cost also scales with prompt size, output size, model selection, retry behavior, and evaluation volume.

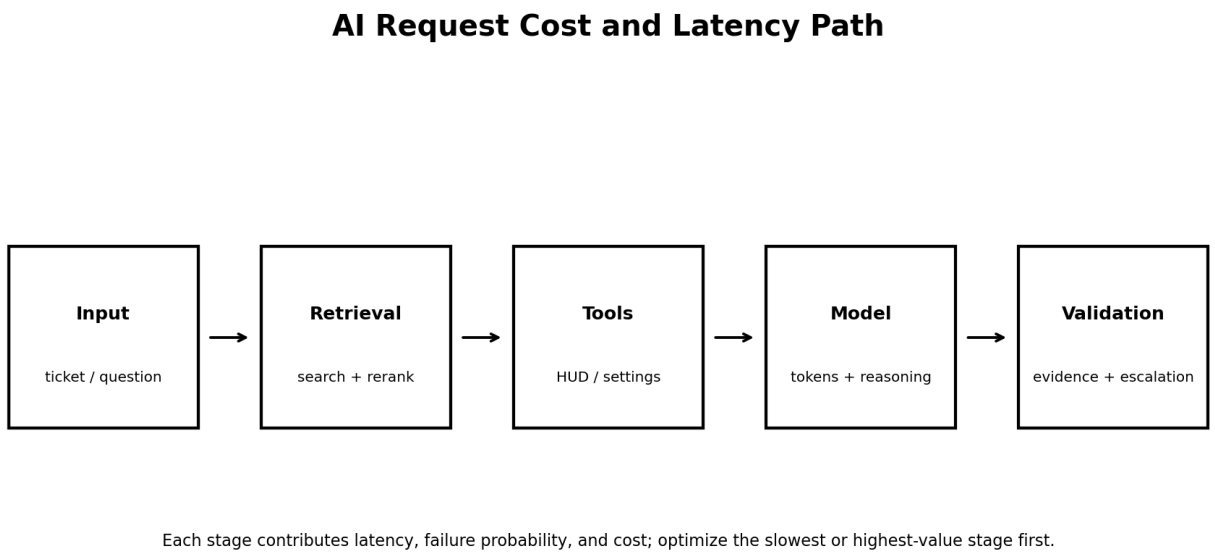


Figure 4. AI request latency and cost accumulate across retrieval, tools, model execution, and validation.

Stage	Performance lever	Quality risk
Triage	Use lightweight deterministic or small-model classification	Misrouting the investigation
Retrieval	Metadata filters, bounded top-k, efficient index	Missing the best evidence
Reranking	Apply only when candidate set justifies cost	Added latency without measurable gain
Tools	Parallelize independent read-only calls with timeouts	Partial evidence or rate-limit failure
Model	Route simple and complex tasks to appropriate models	Quality degradation from over-aggressive cost cutting
Validation	Use structured claims and evidence checks	Skipping validation for speed
Escalation	Stop low-confidence loops and hand off	Excessive human workload if threshold is too strict

The model should not receive the entire knowledge corpus. Retrieval reduces token cost and irrelevant context. Prompt size should be treated as a measurable resource, and repeated evidence should be deduplicated before model execution.

13. Retrieval and Vector-Search Scaling

The current LeaseI implementation uses TF-IDF similarity for a small corpus. That is a reasonable starting point and easy to inspect. A larger deployment would likely require hybrid retrieval, structured metadata filters, reranking, tenant isolation, and index lifecycle management.

- Partition or filter by tenant, program, source type, year, and release before semantic scoring.
- Keep raw tickets separate from staff-confirmed knowledge so trust can influence ranking.
- Measure recall using known-answer datasets rather than relying on subjective examples.
- Track index freshness and failed ingestion.
- Use incremental indexing instead of rebuilding the entire corpus for every change.
- Plan storage and query capacity from document count, chunk count, embedding dimension, and query rate.

Retrieval scaling should not weaken authorization. The correct optimization is to reduce the authorized candidate set early, not retrieve everything and filter sensitive results afterward.

14. Concurrency and Resource Management

Concurrency must be bounded at every shared resource: API workers, database pools, RabbitMQ consumers, file processors, external APIs, and model endpoints. Unlimited parallelism converts a temporary demand spike into platform-wide failure.

Resource	Control
Database connections	Pool maximum, acquisition timeout, separate pools for background and interactive work
Queue consumers	Configurable concurrency and prefetch aligned with transaction time
External APIs	Rate limiter, timeout, circuit breaker, safe cache
LLM endpoints	Per-model concurrency, token budget, queue, fallback
File I/O	Streaming, bounded worker count, object-size limits
CPU-heavy transforms	Worker pool or separate process to avoid blocking API event loop

Workload isolation is especially important. Annual imports or bulk document migrations should not consume all connections needed by customer-facing eligibility and support workflows.

15. Reliability Under Load

Performance problems become reliability problems when timeouts, retries, and queues interact. A slow dependency can cause requests to pile up, consume connections, trigger retries, and produce a cascading failure. Safe systems apply backpressure and shed or defer noncritical work before exhausting shared resources.

- Use bounded queues and define what happens when capacity is full.

- Reject or defer work explicitly rather than allowing indefinite wait.
- Apply retry budgets and exponential backoff with jitter.
- Make operations idempotent before enabling automatic retry.
- Use circuit breakers for failing external dependencies.
- Preserve progress checkpoints for long-running jobs.
- Degrade optional AI or enrichment steps before core deterministic workflows.

16. Observability and Performance Metrics

The useful performance dashboard connects infrastructure metrics to business work. CPU alone does not explain whether annual data is late, a queue is falling behind, or support investigations are timing out.

Layer	Key metrics
Business	records published, certifications affected, documents completed, investigations resolved
API	p50/p95/p99 latency, error rate, timeout rate, request volume
Database	query duration, rows scanned, lock waits, connections, cache hit ratio
Queue	publish rate, delivery rate, depth, oldest-message age, retries, DLQ
ETL	stage duration, throughput, invalid rows, reconciliation variance, checkpoint age
Documents	generation time, object transfer rate, failures, regeneration
AI	retrieval latency, tool latency, model latency, token use, cost, escalation rate
Capacity	utilization, saturation, backlog growth, time to drain, headroom

Percentiles are more useful than averages for interactive latency. Queue age is often more meaningful than queue count because it reveals whether the business deadline is at risk.

17. Performance Testing

Performance tests should model realistic business shapes and validate correctness under load. A synthetic test that repeatedly reads one cached row does not represent an annual import, document migration, or AI investigation.

- Baseline the current system before optimization.
- Use representative data volume and distribution.
- Test steady state, burst, failure, recovery, and retry scenarios.
- Measure the full dependency chain, not only one service.
- Verify business outputs and reconciliation during and after load.
- Test queue drain time and database impact at multiple consumer counts.

- Run AI evaluations with realistic context size and tool latency.
- Set regression thresholds in CI or release validation for critical paths.

18. Incident Diagnosis and Bottleneck Analysis

When a system slows down, I avoid changing multiple components at once. The investigation should identify where time is spent and whether the issue is demand growth, a code regression, a data-distribution change, dependency failure, retry amplification, or resource saturation.

1. Define the affected workflow and time window.

2. Compare current demand and data shape with the previous healthy period.

3. Trace latency across API, queue, worker, database, external API, and model stages.

4. Inspect saturation signals such as connections, locks, CPU, memory, or rate limits.

5. Check recent releases, schema changes, configuration, and index statistics.

6. Form a ranked bottleneck hypothesis and validate with controlled evidence.

7. Contain the load or isolate the workload before risky optimization.

8. After correction, confirm recovery of both technical metrics and business backlog.

The migration and configuration incidents I handled reinforced the same principle: timing correlation is not proof of causation. Performance diagnosis should rely on traces, plans, counts, and preserved evidence.

19. Deployment and Scaling Strategy

Scaling changes should be incremental and reversible. Increasing worker count, adding an index, changing batch size, or introducing a cache can shift pressure to another component. Each change needs a hypothesis, expected metric movement, validation window, and rollback.

Change	Safe rollout
New index	Test plan and write impact, deploy, compare query latency and write cost
Higher worker concurrency	Increase in steps while observing queue drain, DB locks, and errors
Async API workflow	Introduce operation state, dual-run or limited cohort, validate completion and retries
Cache	Start with versioned low-risk data, monitor hit rate and stale-result incidents
Model route change	Shadow or canary evaluation, compare quality, latency, and cost
Partitioning	Plan migration and pruning behavior, validate old and new access patterns

20. Cost Efficiency

Capacity is constrained by money as well as compute. Efficient systems avoid processing unnecessary data, repeating failed work, storing redundant copies, and invoking expensive models for simple tasks.

- Push filters close to the data source.
- Use set-based database operations where they reduce round trips.
- Avoid repeated document parsing and embedding.
- Cache stable public reference data with version-aware keys.
- Route tasks to the least expensive model that meets quality targets.
- Cap retry and agent-loop counts.
- Measure cost per completed business outcome, not only total cloud spend.
- Archive or tier old operational data according to retention and access needs.

21. Trade-offs and Anti-Patterns

Premature distribution

Splitting a simple system into many services adds network, deployment, and observability cost before scale justifies it.

More workers as the default fix

Concurrency can overwhelm the database or external API and reduce total throughput.

Caching without correctness dimensions

A fast stale value can be worse than a slower authoritative lookup in regulated workflows.

Optimizing only averages

Tail latency and oldest backlog age reveal customer pain hidden by averages.

Retries without budgets

Unbounded retry converts dependency failure into self-generated load.

Large transactions for convenience

They increase lock time, rollback cost, and operational risk.

AI context inflation

Sending more documents and longer prompts increases cost and can reduce answer quality.

Benchmarking without business validation

A technically faster pipeline can silently produce wrong counts or decisions.

22. Modernization Roadmap

Phase	Focus	Outcome
1. Baseline	Standard latency, throughput, queue, DB, and AI metrics	Shared evidence and measurable bottlenecks

Phase	Focus	Outcome
2. Protect resources	Timeouts, pool limits, concurrency caps, retry budgets	Reduced cascading failure risk
3. Isolate workloads	Separate interactive and batch resources, prioritized queues	Annual and bulk work cannot starve users
4. Improve execution	Targeted SQL tuning, batching, async jobs, streaming I/O	Higher throughput with current architecture
5. Scale horizontally	Stateless APIs, elastic workers, production queue operations	Capacity increases without single-host dependency
6. Optimize AI	Model routing, retrieval tuning, semantic cache, cost budgets	Lower latency and cost with measured quality
7. Forecast	Trend-based capacity reviews and release load models	Scaling before business deadlines are threatened

The roadmap deliberately improves measurement and resource protection before adding complex infrastructure. Scaling should follow demonstrated demand and bottlenecks.

23. Reusable Capacity Checklists

23.1 New batch workflow

- Expected and peak record volume
- Expansion factor and output volume
- Deadline and acceptable backlog
- Transaction and batch size
- Checkpoint and rerun behavior
- Database and queue impact
- Validation and reconciliation metrics
- Rollback and partial-failure strategy

23.2 New API endpoint

- Latency objective and timeout
- Expected request rate and burst
- Database queries and pool impact
- External dependency budget
- Pagination and payload limit
- Rate and concurrency control
- Cache suitability and invalidation
- Observability and load test

23.3 New AI workflow

- Retrieval corpus and candidate count
- Tool-call count and rate limits
- Model, context, and output token budgets

- End-to-end latency objective
- Concurrency and queue behavior
- Quality gate and escalation threshold
- Trace retention and privacy
- Cost per completed investigation

24. Lessons Learned

- Measure the real bottleneck before selecting a scaling solution.
- Performance targets must reflect the business workflow and deadline.
- Database concurrency can be more important than individual query speed.
- Asynchronous processing improves responsiveness but requires backlog, retry, and idempotency controls.
- Staging and controlled publication make large transformations safer and easier to measure.
- More parallelism is useful only while downstream systems have capacity.
- Correctness, auditability, and recoverability are part of performance in regulated systems.
- AI latency and cost should be engineered across retrieval, tools, models, and validation, not treated as one opaque call.
- Simple architecture with strong measurement is better than complex infrastructure without a proven need.
- A senior engineer scales systems by understanding constraints, trade-offs, and operational behavior, not by adding technology reflexively.

Closing Perspective

Across data processing, RabbitMQ, SQL optimization, migrations, document workflows, and AI systems, the same pattern holds: understand the work, measure the constraint, protect shared resources, validate correctness, and scale incrementally. Capacity planning is not a one-time sizing exercise. It is a continuous engineering practice that connects product demand, system behavior, reliability, and cost.